Software Requirements Specification (SRS)

Project: School Management System (SMS)

Version: 1.0

Date: April 23, 2026

Stack: Go (Gin), React, PostgreSQL

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed description of the School Management System (SMS). It outlines the functional and non-functional requirements, user roles, and system constraints required to build a secure, scalable, and high-performance academic platform.

1.2 Scope

The SMS is a web-based application designed to automate administrative tasks, track student academic progress, and facilitate communication between teachers, parents, and students. The system will feature a Go-based REST API and a React-based frontend, emphasizing security and mobile-ready integration.

1.3 Definitions, Acronyms, and Abbreviations

- **SMS:** School Management System
- **RBAC:** Role-Based Access Control
- **JWT:** JSON Web Token (for authentication)
- **GORM:** Go Object-Relational Mapper
- **Locker:** Digital Student Locker (Secure document storage)

2. Overall Description

2.1 Product Perspective

The SMS is a standalone system that replaces manual paper-based or disjointed legacy software. It provides a centralized database for all institutional data.

2.2 User Classes and Characteristics

1. **System Administrator:** Manages school-wide settings, user accounts, and system audits.
2. **Teacher:** Manages curricula, records attendance, inputs grades, and monitors class performance.
3. **Student:** Views schedules, tracks grades, and manages files in the Digital Locker.
4. **Parent:** Monitors their children's attendance and academic reports.

2.3 Design and Implementation Constraints

- **Backend:** Must use Go with the Gin Gonic framework.
- **Frontend:** Must use React with state management (React Context).
- **Security:** Compliance with OWASP Top 10 (SQLi, XSS, CSRF protection).
- **Timeline:** Full delivery within 2 months (8 weeks).

3. System Features (Functional Requirements)

3.1 Identity & Access Management (IAM)

- **FE-01:** Secure login using JWT.
- **FE-02:** Role-based redirection to specific dashboards (Admin/Teacher/Student).
- **FE-03:** Profile management for all user types.

3.2 Student & Teacher Management

- **FE-04:** Admin can register, update, and archive student/teacher profiles.
- **FE-05:** Automated student enrollment into specific classes and subjects.

3.3 Attendance System

- **FE-06:** Teachers can record daily or subject-wise attendance.
- **FE-07:** Real-time calculation of attendance percentages.
- **FE-08:** Automated alerts sent to parents for student absence.

3.4 Academic Performance & Grading

- **FE-09:** Configurable grading scales (GPA or Letter grades).
- **FE-10:** Bulk grade entry for teachers via a grid-based UI.
- **FE-11:** Generation of digital report cards.

3.5 Digital Student Locker

- **FE-12: Encrypted Storage:** A private section for students to upload certificates, portfolios, and assignments.
- **FE-13: Resource Management:** Ability to keep study materials and textbooks.

- **FE-14: Admin/Teacher View:** Permission-based access for teachers to review student assignments.

3.6 Advanced Analytics & Reporting

- **FE-15:** Visual dashboards showing grade trends (bar charts, line graphs).
- **FE-16:** Comparative analytics (Class A vs. Class B).
- **FE-17:** Exportable reports (PDF/CSV) for administrative audits.

3.7 Notification Engine

- **FE-18:** SMTP-based email notifications for grade releases.
- **FE-19:** Broadcast system for school-wide announcements.

3.8 Finance

- **FE-20:** Payroll system for school staff.
- **FE-21:** Student payment confirmation using bank receipt transaction id(local Ethiopia).
- **FE-20:** Financial transaction tracker for income and expenses(electricity, utilities, etc) and kpi.

4. External Interface Requirements

4.1 User Interfaces

- **Design:** Modern, minimalist UI.
- **Responsiveness:** Fully functional on Desktop and Mobile browsers.
- **UX:** Single Page Application (SPA) behavior for seamless navigation.

4.2 Software Interfaces

- **Database:** PostgreSQL for relational data integrity.
- **Storage:** Local Encrypted Storage for the Digital Locker files.
- **API:** RESTful endpoints.

5. Non-Functional Requirements

5.1 Security

- **Injection:** Mandatory use of GORM parameterized queries.
- **Sensitive Data:** Passwords hashed using bcrypt.
- **Session Management:** Short-lived JWTs with refresh token rotation.

- **XSS/CSRF:** React built-in sanitization and Gin-based CSRF middleware.

5.2 Performance

- **Latency:** API response time < 500ms for standard queries.
- **Concurrency:** Go's Goroutines utilized.

5.3 Reliability

- **Uptime:** Target 97.9% availability.
- **Backups:** Weekly database backups.

5.4 System Constraints

- Web-based system only (no mobile app)
- Requires internet connection
- Built using Go + React

5.5 Future Enhancements

- SMS notifications
- AI-based analytics
- Mobile application
- Online exams

5.6 Scalability

- Modular architecture
- API-first design for mobile apps